Theory of Algorithms – Divide and Conquer

Patrick Marais
Department of Computer Science
University of Cape Town
patrick@cs.uct.ac.za

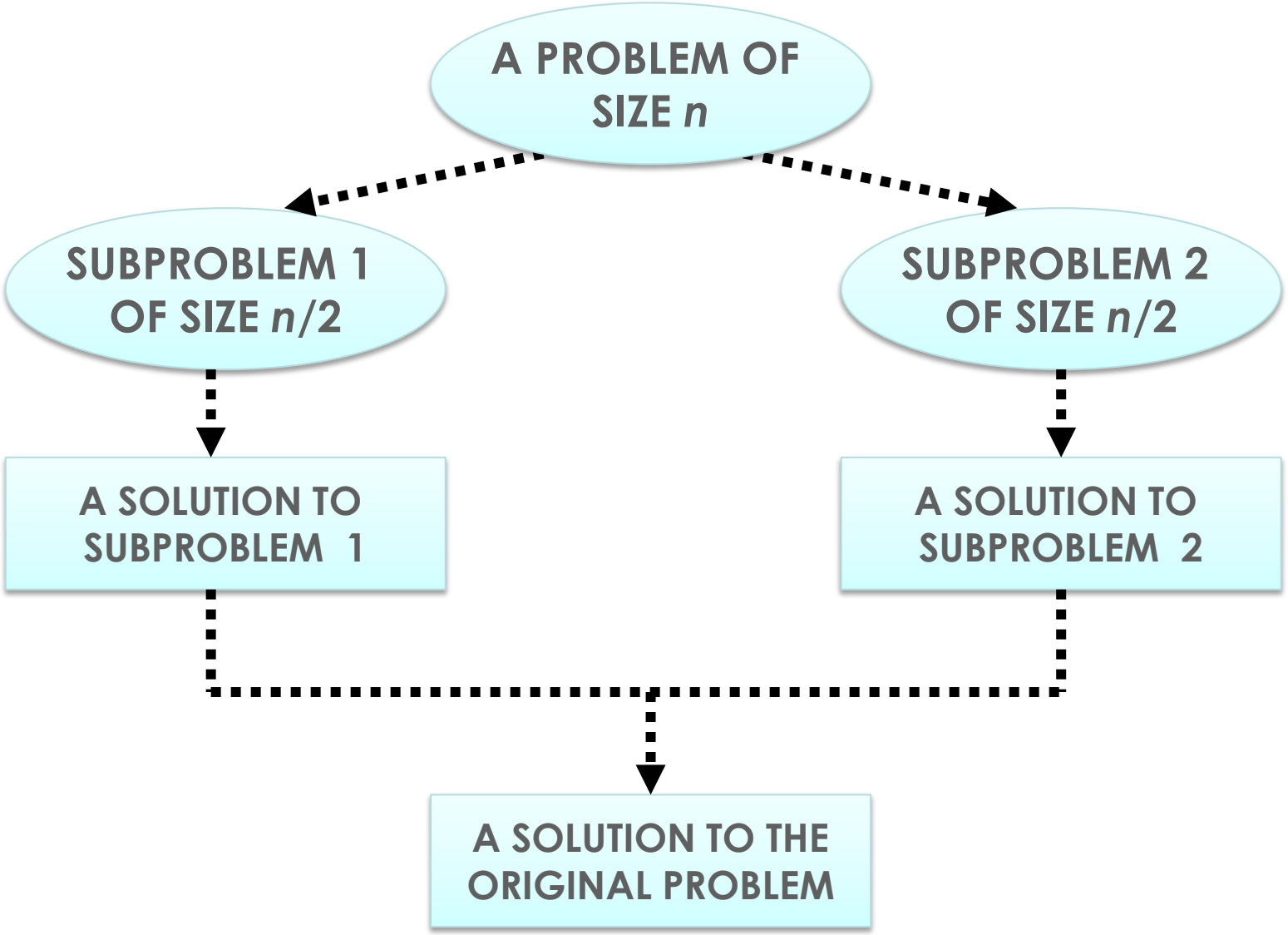




Introduction

- Best known algorithm design strategy:
  1. Divide instance of problem into two or more smaller instances
  2. Solve smaller instances recursively
  3. Obtain solution to original (larger) instance by combining these solutions
- Recurrence relations apply
  - Recurrences are of the form $T(n) = aT(n/b) + f(n)$, $a >= 1$, $b >= 2$
- Simple Example (Addition):
  - $a_0 + \ldots + a_{n-1} = (a_0 + \ldots + a_{n/2-1}) + (a_{n/2} + \ldots a_{n-1})$

Divide and Conquer Illustrated

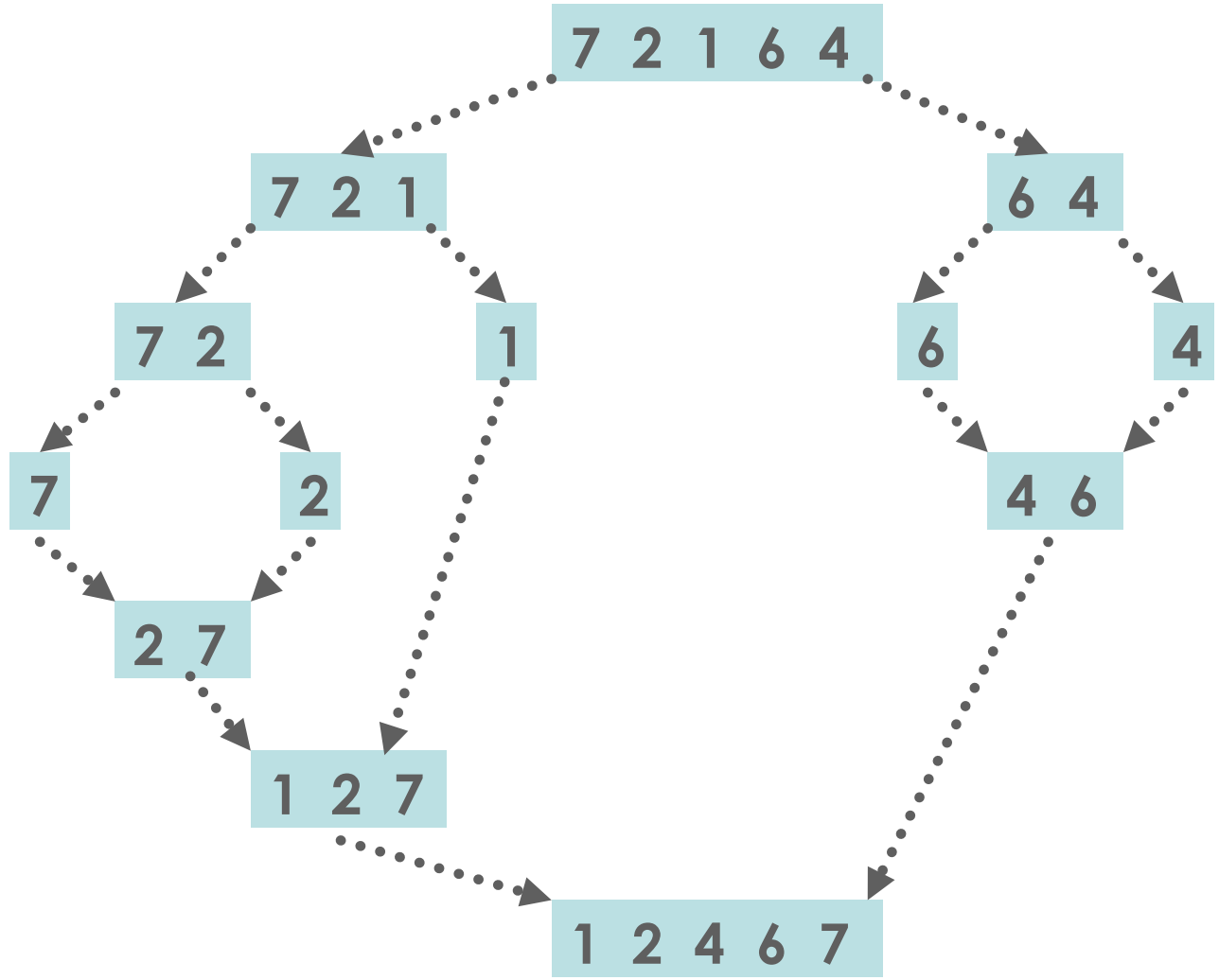


MergeSort

- Algorithm:
  1. Split *A[1..n]* in half and put copy of each half into arrays *B[1.. n/2]* and *C[1.. n/2] (note, if n is odd one array will have one more element)*
  2. Recursively MergeSort arrays B and C
  3. Merge sorted arrays B and C into array A
- Merging:
  - REPEAT until no elements remain in one of B or C
  1. Compare 1st elements in the lists for B and C
  2. Copy smaller into A, incrementing index of corresponding array
  3. Once all elements in one of B or C are processed, copy the remaining unprocessed elements from the other array into A

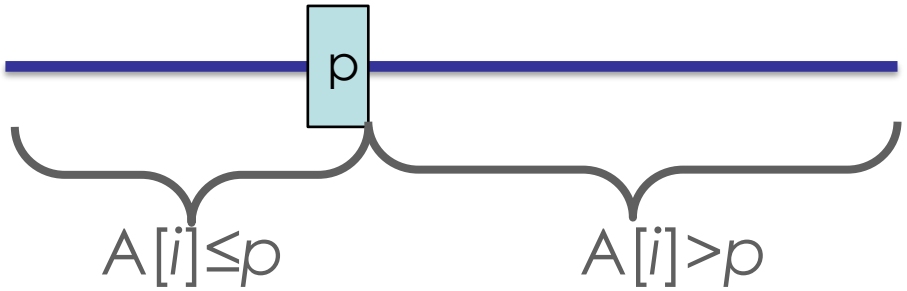
MergeSort Example

- Master Theorem:
  - $T(n) = aT(n/b) + f(n)$, $f(n) \in \Theta(n^d)$
  - $T(n) \in \Theta(n^d \log n)$ if $a = b^d$
- Recurrence (assume $n=2^k$):
  - $C(n) = 2\,C(n/2) + C_{merge}(n)$ *for* $n > 1$, $C(1) = 0$
  - $C_{merge}(n) = n - 1$ in the worst case
- Efficiency (all cases):
  - $a = b^d$ $(2 = 2^1)$ in Master Theorem
    $\Theta(n \log n)$
- Pro: stable (vs quicksort, heapsort)
- Con: uses extra space $\Theta(n)$



QuickSort (Reminder)

- Select a *pivot* (partitioning element) – this is A[0] usually
- Rearrange the list into two sub-lists
  - All elements positioned before the pivot are ≤ the pivot
  - Those positioned after the pivot are > the pivot
  - Requires a pivoting algorithm
- Exchange the pivot with the last element in the first sub-list
  - The pivot is now in its final position
- QuickSort the two sub-lists

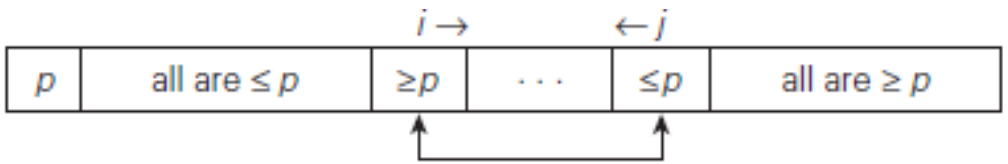


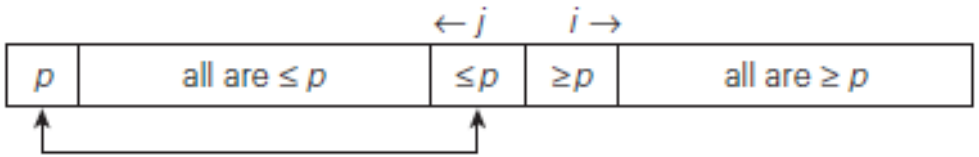
ALGORITHM *Quicksort*$(A[l..r])$

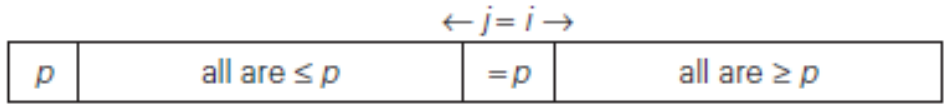
//Sorts a subarray by quicksort
//Input: Subarray of array $A[0..n-1]$, defined by its left and right
// indices l and r
//Output: Subarray $A[l..r]$ sorted in nondecreasing order
if $l < r$
  $s \leftarrow$*Partition*$(A[l..r])$ //s is a split position
  Quicksort$(A[l..s-1])$
  Quicksort$(A[s+1..r])$

ALGORITHM *HoarePartition*$(A[l..r])$

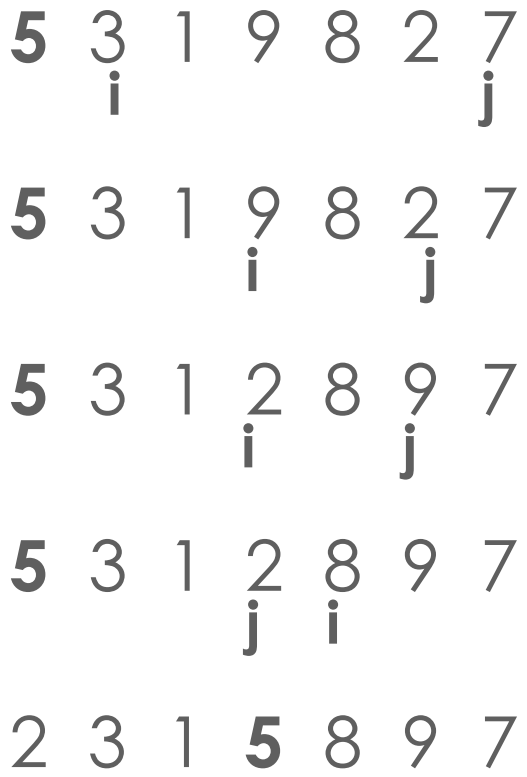
//Partitions a subarray by Hoare's algorithm, using the first element
// as a pivot
//Input: Subarray of array $A[0..n-1]$, defined by its left and right
// indices l and r $(l < r)$
//Output: Partition of $A[l..r]$, with the split position returned as
// this function's value
$p \leftarrow A[l]$
$i \leftarrow l$; $j \leftarrow r+1$
repeat
  repeat $i \leftarrow i+1$ **until** $A[i] \geq p$
  repeat $j \leftarrow j-1$ **until** $A[j] \leq p$
  swap$(A[i], A[j])$
until $i \geq j$
swap$(A[i], A[j])$ //undo last swap when $i \geq j$
swap$(A[l], A[j])$
return j



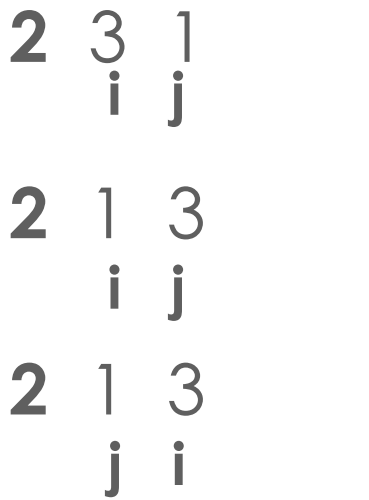




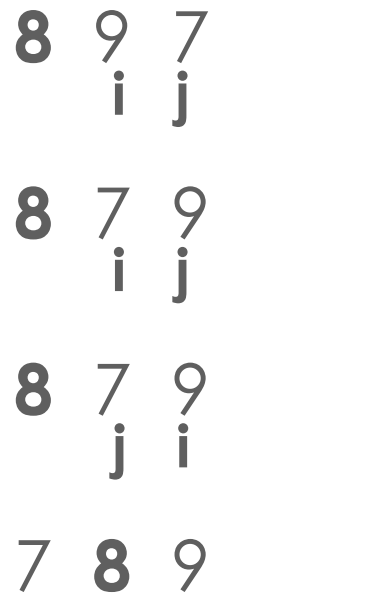
QuickSort Example



Recursive Call Quicksort (2 3 1) & Quicksort (8 9 7)

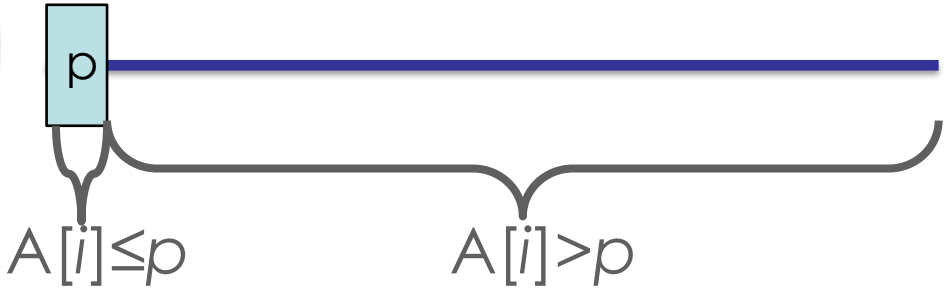


Recursive Call Quicksort (1) & Quicksort (3)



Recursive Call Quicksort (7) & Quicksort (9)

Worst-Case Efficiency of QuickSort

- In the worst case all splits are completely skewed
  - 

- For instance, an already sorted list (ascending order)!
- One subarray is empty, other reduced by only one:
  - Make *n+1* comparisons
  - Exchange pivot with itself
  - Quicksort left = Ø, right = A[1..n-1]
  - $C_{worst} = (n+1) + n + \ldots + 3 = (n+1)(n+2)/2 - 3 = \Theta(n^2)$

General Efficiency of QuickSort

- While worst case is $\Theta(n^2)$, best case (split in the middle) is $\Theta(n \log n)$ and average case (random split) is $\Theta(n \log n)$
- Improvements (in combination 20-25% faster):
  - Better pivot selection: median of three partitioning avoids worst case in sorted lists
  - Switch to Insertion Sort on small subsets
  - Elimination of recursion
- Considered the method of choice for internal sorting for large lists ($n \geq 10000$)

Efficient matrix multiplication: Strassen's method

- Brute force multiplication of two n x n matrices is $\Theta(n^3)$ ops,
- Can solve matrix multiplication *recursively* (via block matrix operations) - still $\Theta(n^3)$, **but** can then trade multiplies for adds – Strassen's method

$$\begin{bmatrix} c_{00} & c_{01} \\ c_{10} & c_{11} \end{bmatrix} = \begin{bmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{bmatrix} * \begin{bmatrix} b_{00} & b_{01} \\ b_{10} & b_{11} \end{bmatrix}$$
$$= \begin{bmatrix} m_1 + m_4 - m_5 + m_7 & m_3 + m_5 \\ m_2 + m_4 & m_1 + m_3 - m_2 + m_6 \end{bmatrix}$$

$$\begin{aligned} m_1 &= (a_{00} + a_{11}) * (b_{00} + b_{11}), \\ m_2 &= (a_{10} + a_{11}) * b_{00}, \\ m_3 &= a_{00} * (b_{01} - b_{11}), \\ m_4 &= a_{11} * (b_{10} - b_{00}), \\ m_5 &= (a_{00} + a_{01}) * b_{11}, \\ m_6 &= (a_{10} - a_{00}) * (b_{00} + b_{01}), \\ m_7 &= (a_{01} - a_{11}) * (b_{10} + b_{11}). \end{aligned}$$

$$\left[\begin{array}{c|c} C_{00} & C_{01} \\ \hline C_{10} & C_{11} \end{array}\right] = \left[\begin{array}{c|c} A_{00} & A_{01} \\ \hline A_{10} & A_{11} \end{array}\right] * \left[\begin{array}{c|c} B_{00} & B_{01} \\ \hline B_{10} & B_{11} \end{array}\right].$$

- Multiplies: M(n) = 7M(n/2), for n > 1, M(1) = 1
- Adds: A(n) = 7A(n/2) + 18(n/2)^2, for n > 1, A(1) = 0
- Both M(n), A(n) ∈ $\Theta(n^{\log_2 7}) = \Theta(n^{2.81})$ (via Master Theorem)

Source: http://www.cs.mcgill.ca/~pnguyen/251F09/matrix-mult.pdf

```
STRASSEN(A, B):
    1. If n == 1  Output  A11 × B11
    2. Else
    3.          Split matrices into 8 n/2 X n/2 parts: A11 , B11 , . . . , A22 , B22
    4.          P1 = Strassen(A11, B12 − B22 )
    5.          P2 = Strassen(A11 + A12 , B22 )
    6.          P3 = Strassen(A21 + A22 , B11 )
    7.          P4 = Strassen(A22 , B 21 − B11 )
    8.          P5 = Strassen(A11 + A22 , B11 + B22 )
    9.          P6 = Strassen(A12 − A22 , B21 + B22 )
    10          P7 = Strassen(A11 − A21 , B11 + B12 )
    11.         C 11 = P5 + P4 − P2 + P6
    12.         C 12 = P1 + P2
    13.         C 21 = P3 + P4
    14.         C 22 = P1 + P5 − P3 − P7
    15.         Output C
```

- Better algorithms now exist – e.g. Coppersmith and Winograd, $O(n^{2.376})$ or even faster
- But many methods only relevant for 'galactic problems' (Huge matrices!)

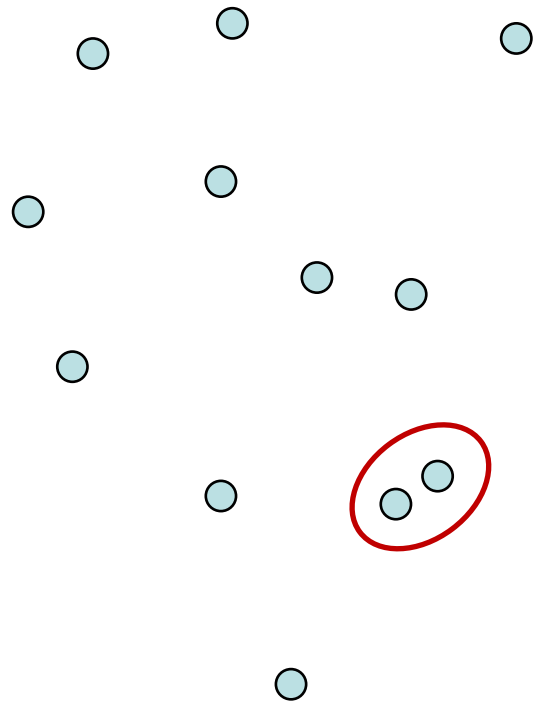
Theory of Algorithms – Divide and Conquer (Geometric Problems)





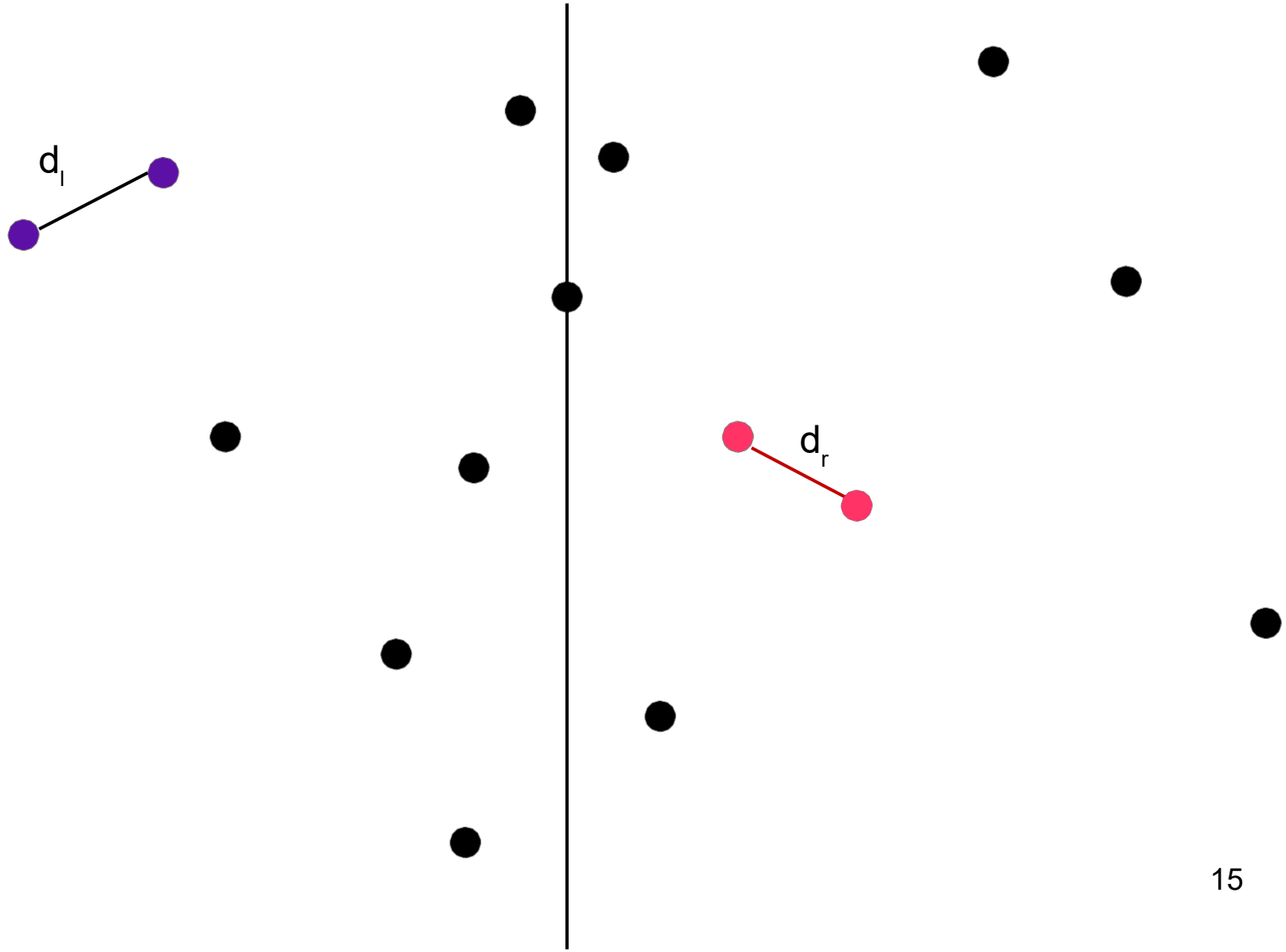
Closest Pair by Divide and Conquer

1. **Sort** points according to their x-coordinates
2. **Split** the set of points into two equal-sized subsets by a vertical line $x=x_{median}$
3. **Solve** the problem recursively in the left and right subsets, so that we get the left-side and right-side minimum distances d_l and d_r. $d_{min}=\min(d_l, d_r)$
4. **Straddle**. Find the minimal distance in the set S of points of width 2d around the vertical line. Update d_{min} if necessary



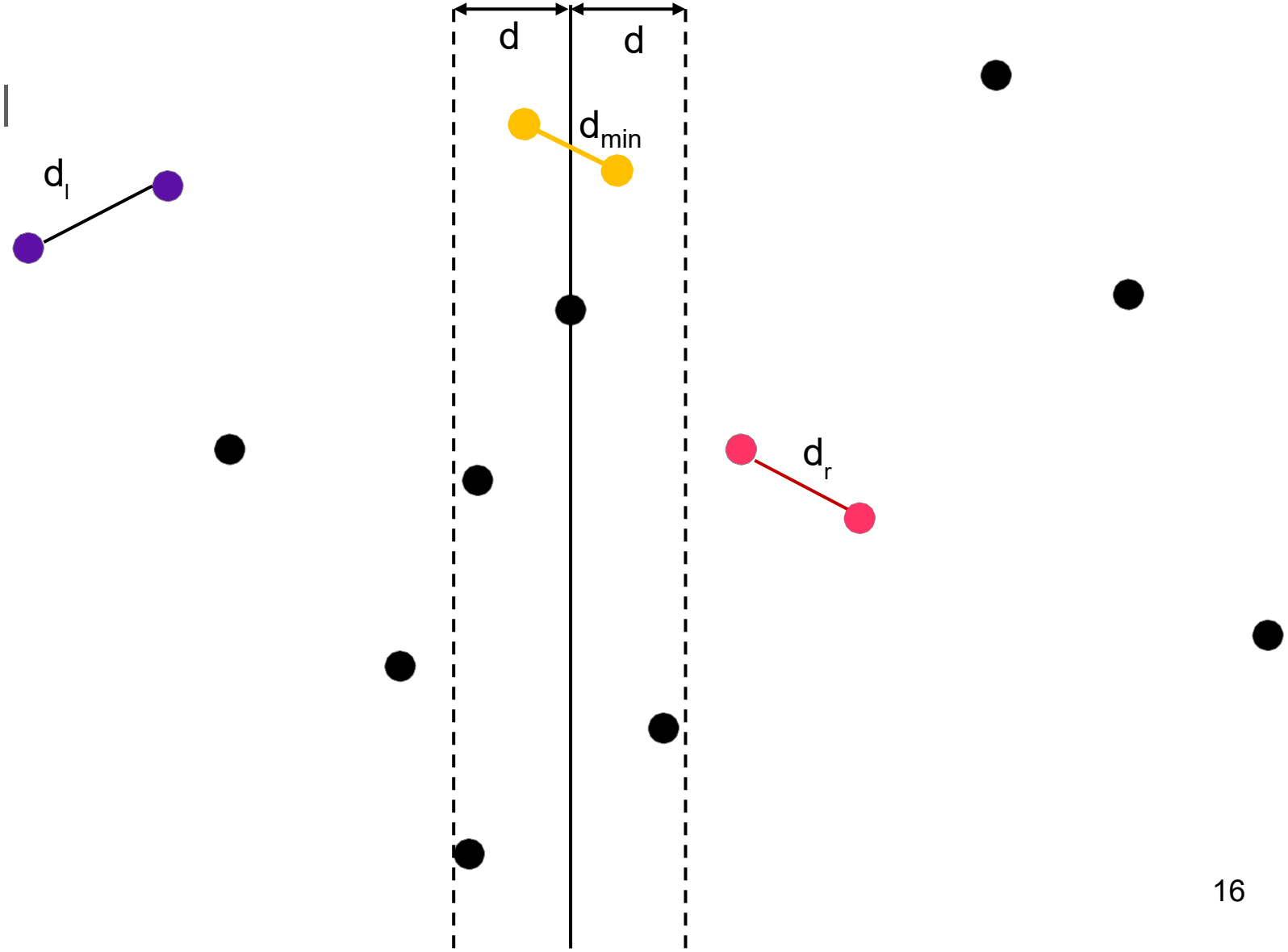
Closest Pair Illustrated: Split & Solve

- Recursively find shortest pair in each half
- Let $d_{min} = \min(d_l, d_r)$
  - $d_{min} = d_r$ here



Closest Pair Illustrated: Straddle

- Search for potential new d_{min} among points within d of dividing line
- Efficiency:
  - T(n) = 2T(n/2) + f(n), f(n) ∈ Θ(n) (merge + split)
  - a=2, b=2, d=1
  - $a = b^d$ in Master Theorem
  - → Θ(n log n)
  - Same cost as pre-sorting step

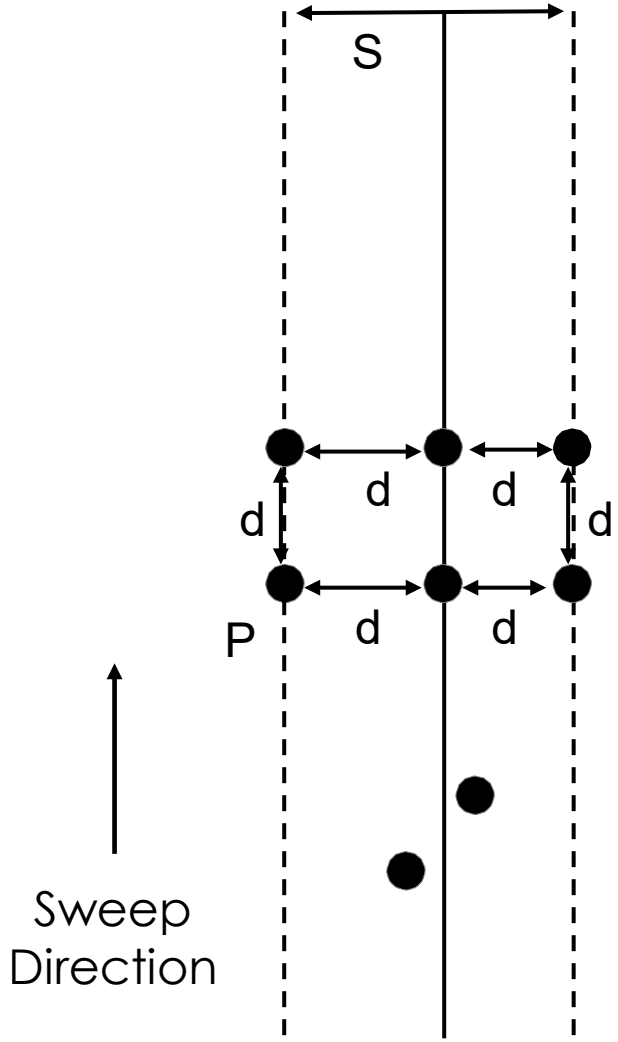


Limits in the Straddle Zone

- Initially hard to see how this can differ in efficiency from brute force
  - What if S has almost as many points in it as the non-S part?
  - Then for each point in S we compare it to **ALL** points in front of it by Brute Force:
  - for i = 0 to n – 2:
    for j = i + 1 to n-1: etc
  - Efficiency – $\Theta(n^2)$
- But there is actually a property of S that dramatically limits the number of points we have to look at

Straddle

- Assuming point in S are sorted by increasing y. We need only compare against the next 5 points
- Proved via "packing argument"
  - Given the restriction that points to left and right of the dividing line must be at least d = d_{min} apart
  - Configuration on the right represents the *closest packing*
  - It is not possible for a 6th point to be closer to P than d in the sweep direction

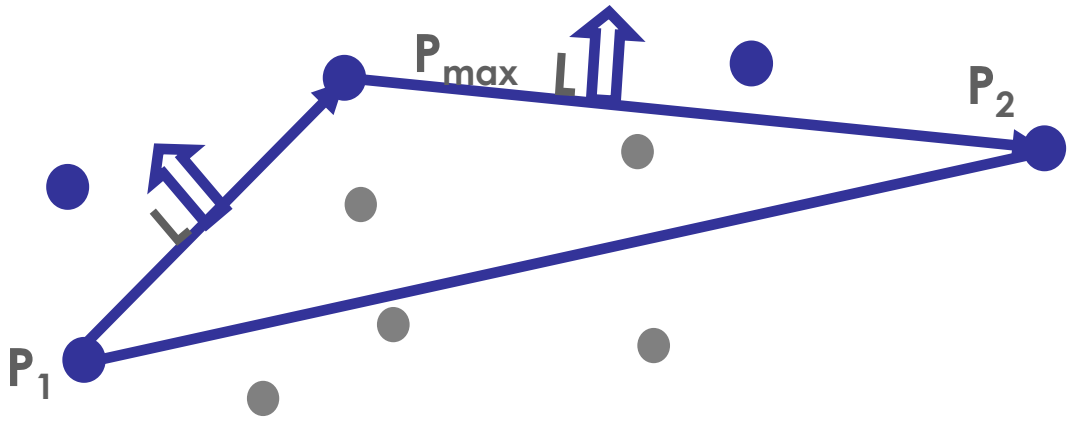


ALGORITHM *EfficientClosestPair*(P, Q)

```
//Solves the closest-pair problem by divide-and-conquer
//Input: An array P of n ≥ 2 points in the Cartesian plane sorted in
//       nondecreasing order of their x coordinates and an array Q of the
//       same points sorted in nondecreasing order of the y coordinates
//Output: Euclidean distance between the closest pair of points
if n ≤ 3
    return the minimal distance found by the brute-force algorithm
else
    copy the first ⌈n/2⌉ points of P to array P_l
    copy the same ⌈n/2⌉ points from Q to array Q_l
    copy the remaining ⌊n/2⌋ points of P to array P_r
    copy the same ⌊n/2⌋ points from Q to array Q_r
    d_l ← EfficientClosestPair(P_l, Q_l)
    d_r ← EfficientClosestPair(P_r, Q_r)
    d ← min{d_l, d_r}
    m ← P[⌈n/2⌉ − 1].x
    copy all the points of Q for which |x − m| < d into array S[0..num − 1]
    dminsq ← d^2
    for i ← 0 to num − 2 do
        k ← i + 1
        while k ≤ num − 1 and (S[k].y − S[i].y)^2 < dminsq
            dminsq ← min((S[k].x − S[i].x)^2 + (S[k].y − S[i].y)^2, dminsq)
            k ← k + 1
return sqrt(dminsq)
```

QuickHull

- Solve the Convex Hull problem in an approach reminiscent of QuickSort
- Worst case: $\Theta(n^2)$, but average is $\Theta(n)$
- Solution
  - Identify leftmost and rightmost points P_1 and P_2
  - Compute upper hull:
    - Find point P_{max} that is farthest away from line P_1P_2
    - Quickhull the points to the left of line P_1P_{max}
    - Quickhull the points to the left of line $P_{max}P_2$
  - Similarly compute lower hull



Triangle Area = 0.5*|det(P1,P2,Pmax)|

$$\begin{vmatrix} x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \\ x_3 & y_3 & 1 \end{vmatrix} = x_1y_2 + x_3y_1 + x_2y_3 - x_3y_2 - x_2y_1 - x_1y_3$$

QuickHull Example

